Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen 2 — Algoritmos, Estructuras de Datos, Compiladores, Memoria y Sistemas Operativos

Partes III y IV: Complejidad Asintótica, Grafos, Programación Dinámica, Arquitectura de Cachés, Compiladores SSA y Sistemas Operativos

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I PARTE III. Algoritmos, Estructuras de Datos y Computación 1

1. Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado 2
1.1. Definición formal de O, Omega y Theta, y Teorema Maestro para recurrencias 2
1.2. Análisis amortizado por método de potencial y vectores dinámicos 2

2. Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes 6
2.1. Búsqueda binaria, invariantes de bucle y prevención del desbordamiento de punto medio . 6
2.2. Cota inferior por árbol de decisión, Mergesort, Quicksort, Heapsort y ordenación lineal . . 6

3. Divide y vencerás, algoritmos voraces, programación dinámica y backtracking 10
3.1. Divide y vencerás y condición de elección voraz (Greedy Choice Property) 10
3.2. Programación dinámica (Bellman) y búsqueda exhaustiva con poda (Backtracking) 10

4. Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing 14
4.1. BFS, DFS, componentes conexas, Dijkstra, Bellman-Ford, Kruskal y Prim 14
4.2. Árboles equilibrados (AVL, Rojo-Negro, B+ Trees) y tablas hash de alto rendimiento . . . 15

5. Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span) 18
5.1. Estructuras persistentes (path copying) y filtros probabilísticos (Bloom Filter) 18
5.2. Algoritmos paralelos: modelo Work-Span, ley de Brent y reducciones paralelas 19

II PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos 22

6. Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado 23
6.1. Líneas de caché, coherencia MESI, false sharing y diseño orientado a datos 23
6.2. Alineamiento en memoria, ABI de plataforma y enlazadores (ELF/Mach-O) 24

7. Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código 27
7.1. Front-end del compilador: análisis léxico, sintáctico (descendente recursivo y Pratt) y semántico 27
7.2. Representación intermedia SSA, optimizaciones e instrucción/asignación de registros . . . 28

8. Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S 31
8.1. Modo usuario vs modo kernel, llamadas al sistema, procesos, hilos y planificación 31
8.2. Memoria virtual, tablas de páginas, TLB, sistemas de archivos y E/S no bloqueante . . . 32

A. Notación 35

B. Hoja de fórmulas **36**

C. Soluciones **37**
C.1. Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado . . . 37
C.2. Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes . . . 38
C.3. Divide y vencerás, algoritmos voraces, programación dinámica y backtracking . . . 40
C.4. Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing . . . 41
C.5. Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span) . . . 43
C.6. Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado . . . 45
C.7. Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código . . . 46
C.8. Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S . . . 48

D. Auditoría de fuentes **50**

E. Plan de estudio **52**

Parte I

PARTE III. Algoritmos, Estructuras de Datos y Computación

Capítulo 1

Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

Proporcionar las herramientas matemáticas exactas para predecir cómo escalan el tiempo de ejecución y el consumo de memoria de cualquier algoritmo cuando el tamaño de entrada n crece varios órdenes de magnitud, demostrando cotas superiores, inferiores y costes amortizados.

Consejo

En un vector dinámico, si en lugar de multiplicar la capacidad por una constante ($\times 2$ o $\times 1{,}5$) se incrementara sumando una cantidad fija $+K$ en cada llenado, el coste de n inserciones degeneraría a $\Theta(n^2)$: la progresión geométrica es indispensable para mantener `push` en $O(1)$ amortizado.

Definición

Método del Potencial (Análisis Amortizado): dada una secuencia de m operaciones con costes reales $c_1, \ldots, c_m$ sobre estados $D_0, \ldots, D_m$ y una función de potencial $\Phi(D_i) \geq \Phi(D_0)$, la suma de costes amortizados acota superiormente el coste real total: $\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i - (\Phi(D_m) - \Phi(D_0)) \leq \sum_{i=1}^{m} \hat{c}_i$.

Autoficha

1. ¿Qué diferencia existe entre decir que un algoritmo es $O(n^2)$ y decir que es $\Theta(n^2)$? 2. ¿Qué diferencia fundamental separa la complejidad en el caso promedio de la complejidad amortizada? 3. ¿Por qué duplicar la capacidad de un array dinámico garantiza coste amortizado $\hat{c}_i \leq 3$?

1.1 Definición formal de O, Omega y Theta, y Teorema Maestro para recurrencias

El análisis de algoritmos mide el crecimiento del número de operaciones elementales $T(n)$ y de celdas de memoria $S(n)$ en función del tamaño de entrada n. Dadas dos funciones no negativas $f(n)$ y $g(n)$: 1. Cota superior asintótica ($f(n) \in O(g(n))$): existen constantes $c > 0$ y $n_0 \geq 1$ tales que $0 \leq f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. 2. Cota inferior asintótica ($f(n) \in \Omega(g(n))$): existen $c > 0$ y $n_0 \geq 1$ tales que $0 \leq c \cdot g(n) \leq f(n)$ para todo $n \geq n_0$. 3. Orden asintótico exacto ($f(n) \in \Theta(g(n))$): se cumple simultáneamente $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$. Para algoritmos recursivos de divide y vencerás cuya recurrencia tiene la forma $T(n) = aT(n/b) + f(n)$ con $a \geq 1$ y $b > 1$, el Teorema Maestro compara $f(n)$ con la hoja crítica $n^{\log_b a}$: (Caso 1) si $f(n) = O(n^{\log_b a - \varepsilon})$, entonces $T(n) = \Theta(n^{\log_b a})$; (Caso 2) si $f(n) = \Theta(n^{\log_b a})$, entonces $T(n) = \Theta(n^{\log_b a} \log n)$ (como ocurre en Mergesort con $a = 2, b = 2, f(n) = \Theta(n) \Rightarrow \Theta(n \log n)$); (Caso 3) si $f(n) = \Omega(n^{\log_b a + \varepsilon})$ y satisface regularidad, entonces $T(n) = \Theta(f(n))$.

1.2 Análisis amortizado por método de potencial y vectores dinámicos

No debe confundirse la complejidad en el caso promedio (que asume una distribución probabilística sobre las entradas) con la complejidad amortizada: el análisis amortizado garantiza de forma determinista el coste promedio por operación sobre cualquier secuencia de m operaciones en el peor caso, sin ninguna hipótesis probabilística. En el método de la función de potencial de Tarjan, definimos una función $\Phi(D_i) \geq \Phi(D_0)$ sobre el estado D_i de la estructura de datos, y el coste amortizado de la i-ésima operación es $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Como muestra la figura 1, en un vector dinámico (`Vec<T>` en Rust, `std::vector`

en C++, `list` en Python) que duplica su capacidad cap cuando el tamaño len alcanza cap, tomamos $\Phi = 2 \cdot \text{len} - \text{cap}$. En una inserción sin redimensionado ($c_i = 1, \Delta\text{len} = 1, \Delta\text{cap} = 0$), el coste amortizado es $\hat{c}_i = 1 + 2 = 3$. En una inserción con redimensionado de k a $2k$ ($c_i = k + 1$), el potencial cae de $\Phi_{i-1} = 2k - k = k$ a $\Phi_i = 2(k + 1) - 2k = 2$, por lo que $\hat{c}_i = (k + 1) + (2 - k) = 3 = \Theta(1)$.

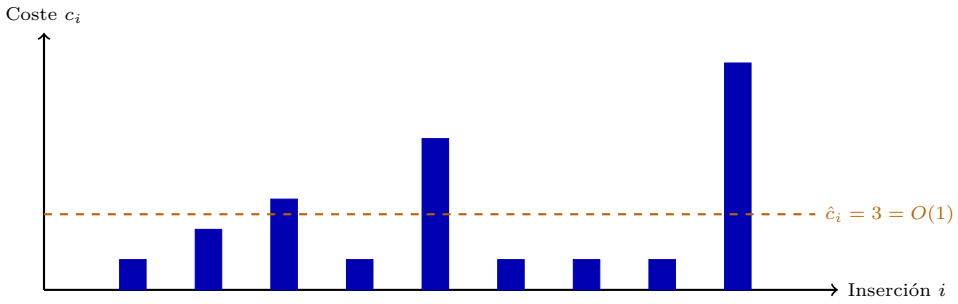


Figura 1. Evolución de capacidad y tamaño en un vector dinámico con factor de crecimiento 2x: las reasignaciones en potencias de 2 quedan amortizadas a coste O(1) por inserción.

Problema resuelto

Enunciado Un vector dinámico comienza con capacidad inicial $\text{cap}_0 = 1$ y duplica su capacidad cada vez que se llena ($1 \to 2 \to 4 \to 8 \to 16 \to 32$). Si se insertan secuencialmente $n = 17$ elementos y cada inserción cuesta 1 escritura más el número de elementos copiados durante el redimensionado ($1 + 2 + 4 + 8 + 16$ copias acumuladas), calcule el coste total exacto de escrituras realizadas.

Datos

- $n = 17$ — Número de elementos insertados
- $C_{copy} = 31$ — Copias de redimensionado (1+2+4+8+16)

Incógnita Coste total de operaciones de escritura sobre las 17 inserciones

Modelo físico

Suma geométrica de copias en redimensionamiento duplicado de un vector dinámico

$$C_total = n + C_copy$$

Hipótesis

- Capacidad inicial 1 y duplicación exacta al insertar cuando len == cap

Desarrollo

1. Sumamos los elementos copiados en las 5 expansiones (al pasar de capacidad 1 a 2, 4, 8, 16 y 32).

$$C_copy = 1 + 2 + 4 + 8 + 16$$

$$((((1 + 2) + 4) + 8) + 16)$$

2. Sumamos las 17 escrituras directas de cada nuevo elemento.

$$C_total = n + C_copy$$

$$(17 + 31)$$

Resultado

48

Interpretación Las 17 inserciones requieren 48 escrituras en total (menos de 3 * 17 = 51), confirmando la cota teórica del método de potencial (coste amortizado <= 3 por inserción).

Comprobación de los pasos

- Paso 1: 31 — reproducido
- Paso 2: 48 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En sistemas de tiempo real estricto (hard real-time), el pico puntual O(n) del redimensionado obliga a pre-reservar con `with_capacity`.

Errores frecuentes

- Confundir el peor caso de una operación aislada O(n) con el coste de una secuencia de n operaciones, que es O(n) total y no $O(n^2)$.

Ejercicios

1 [PRB-0003] (Básico, Conceptual) [VOL02-C01-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0004] (Básico, Numérico) [VOL02-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0005] (Intermedio, Interpretación) [VOL02-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0006] (Intermedio, Aplicación) [VOL02-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(14 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0007] (Intermedio, Identificación de errores) [VOL02-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0008] (Avanzado, Desarrollo por pasos) [VOL02-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0009] (Avanzado, Comparación) [VOL02-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0010] (Avanzado, Comprobación de hipótesis) [VOL02-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0011] (Nivel de examen, Optimización) [VOL02-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0012] (Nivel de examen, Demostración) [VOL02-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0013] (Reto, Diseño) [VOL02-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0014] (Reto, Estudio de parámetros) [VOL02-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 2

Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

Dominar los algoritmos de búsqueda y ordenación desde sus demostraciones formales de corrección mediante invariantes de bucle hasta la cota teórica de información $\Omega(n \log n)$ y las implementaciones híbridas de producción (Timsort, pdqsort, Radix Sort).

Consejo

En las bibliotecas estándar modernas, elija la ordenación estable (`slice::sort` en Rust, Timsort en Python `sorted`, `std::stable_sort` en C++) cuando ordene registros compuestos o secuencias parcialmente ordenadas, y la ordenación inestable (`sort_unstable` / `pdqsort`) para tipos primitivos donde desee evitar la reserva de memoria auxiliar.

Definición

Cota Inferior de Ordenación por Comparación y Estabilidad: ningún algoritmo que ordene n elementos mediante comparaciones binarias puede realizar menos de $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaciones en el peor caso. Un algoritmo de ordenación es estable si para todo par de índices $i < j$ con claves equivalentes $k(A[i]) = k(A[j])$, el elemento $A[i] precede a $A[j] en la salida.

Autoficha

1. ¿Cómo se demuestra que el árbol de decisión de cualquier algoritmo de ordenación por comparación tiene altura al menos $\log_2(n!)$? 2. ¿Por qué `mid = low + (high - low) / 2` evita el desbordamiento entero de `(low + high) / 2`? 3. ¿Por qué Introsort/pdqsort combina Quicksort, Insertion Sort para subvectores pequeños y Heapsort como salvaguarda?

2.1 Búsqueda binaria, invariantes de bucle y prevención del desbordamiento de punto medio

La búsqueda binaria reduce un espacio ordenado de n elementos a la mitad en cada comparación, alcanzando complejidad $\Theta(\log_2 n)$ en el peor caso. Pese a su brevedad, como documentó Jon Bentley en Programming Pearls, la mayoría de las implementaciones improvisadas contienen defectos en las condiciones de parada o en el cálculo del punto medio: escribir `mid = (low + high) / 2` con enteros de ancho fijo provoca desbordamiento aritmético cuando `low + high` supera `INT_MAX` (defecto que permaneció veinte años oculto en `java.util.Arrays.binarySearch`). La forma correcta es `mid = low + (high - low) / 2`. Para demostrar formalmente la corrección de un bucle (inicialización, mantenimiento y terminación), se trabaja sobre un intervalo semiabierto `[low, high)` con el invariante: para todo índice $i <$ low se cumple $A[i] <$ target y para todo $j \geq$ high se cumple $A[j] \geq$ target. En cada iteración la longitud `high - low` decrece estrictamente (función de cota o variante de bucle), garantizando terminación en $\lceil \log_2(n+1) \rceil$ pasos.

2.2 Cota inferior por árbol de decisión, Mergesort, Quicksort, Heapsort y ordenación lineal

Todo algoritmo de ordenación basado exclusivamente en comparaciones entre pares de elementos puede modelarse como un árbol binario de decisión cuyas hojas contienen las $n!$ permutaciones posibles de la entrada. Un árbol binario de altura h posee a lo sumo 2^h hojas; por tanto, $2^h \geq n!$, y aplicando la

aproximación de Stirling ($\log_2(n!) = n\log_2 n - n\log_2 e + \Theta(\log n)$) se obtiene el teorema fundamental: cualquier algoritmo de ordenación por comparación requiere $h \geq \lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaciones en el peor caso. Mergesort alcanza $\Theta(n \log n)$ garantizado y es estable (preserva el orden relativo de claves iguales), empleando $\Theta(n)$ memoria auxiliar; Heapsort logra $\Theta(n \log n)$ en el peor caso con memoria auxiliar $O(1)$ in-place sobre un montículo binario implícito, pero no es estable y presenta peor localidad de caché que Quicksort; Quicksort con pivote mediano o aleatorio logra $\Theta(n \log n)$ esperado con excelente localidad espacial en caché L1 (y en su variante híbrida Introsort/pdqsort empleada en Rust `sort_unstable` y C++ `std::sort`, conmuta a Heapsort si la profundidad recursiva supera $2\lfloor \log_2 n \rfloor$, garantizando $O(n \log n)$ en el peor caso). Cuando las claves son enteros de w bits agrupados en dígitos de base R, Counting Sort y Radix Sort evitan comparaciones directas y ordenan en tiempo lineal $\Theta(d(n + R))$.

```
fn lower_bound(slice: &[i64], target: i64) -> usize {
    let mut low = 0usize;
    let mut high = slice.len();
    while low < high {
        let mid = low + (high - low) / 2;
        if slice[mid] < target {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    low
}

fn main() {
    let data = [2, 5, 5, 5, 9, 14];
    assert_eq!(lower_bound(&data, 5), 1);
    assert_eq!(lower_bound(&data, 6), 4);
}
```

Problema resuelto

Enunciado Determine el número mínimo teórico de comparaciones binarias en el peor caso necesarias para ordenar $n = 5$ elementos distintos sabiendo que existen $5! = 120$ permutaciones posibles y que $2^6 = 64 < 120 \leq 2^7 = 128$, verificando además la diferencia $$2^7 - 5!$$.

Datos

- $n = 5$ — Número de elementos a ordenar
- $P = 120$ — Número de permutaciones 5!
- $L_7 = 128$ — Número máximo de hojas en altura $h = 7(2^7)$

Incógnita Margen de hojas sobrantes $2^7 - 5!$ que demuestra que h = 7 es el mínimo entero tal que $2^h >= 5!$

Modelo físico

Cota de información en árbol binario de decisión (2^h >= n!)

$$margin = 2^7 - fact_5$$

Hipótesis

- Cada comparación binaria divide el conjunto de permutaciones candidatas en a lo sumo dos subárboles

Desarrollo

1. Calculamos el factorial 5! = 120 permutaciones que deben aparecer como hojas alcanzables.

$$fact_5 = 5 * 4 * 3 * 2 * 1$$

$$((((5 \cdot 4) \cdot 3) \cdot 2) \cdot 1)$$

2. Comprobamos que $2^7 = 128$ cubre las 120 permutaciones con un margen de 8 hojas.

$$margin = 2^7 - fact_5$$

$$(128 - 120)$$

Resultado

$$8$$

Interpretación Con 6 comparaciones solo se distinguen a lo sumo $2^6 = 64$ permutaciones (< 120); por tanto, se necesitan al menos h = 7 comparaciones en el peor caso para ordenar 5 elementos, dejando un margen de 8 hojas.

Comprobación de los pasos

- Paso 1: 120 — reproducido
- Paso 2: 8 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Para n grandes, la cota de información ceil(log2(n!)) es necesaria pero en ciertos n específicos el mínimo exacto de Ford-Johnson requiere 1 comparación adicional.

Errores frecuentes

- Confundir n * log2(n) directo con log2(n!), que para n pequeños difiere significativamente.

Ejercicios

1 [PRB-0017] (Básico, Conceptual) [VOL02-C02-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0018] (Básico, Numérico) [VOL02-C02-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0019] (Intermedio, Interpretación) [VOL02-C02-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0020] (Intermedio, Aplicación) [VOL02-C02-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0021] (Intermedio, Identificación de errores) [VOL02-C02-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0022] (Avanzado, Desarrollo por pasos) [VOL02-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0023] (Avanzado, Comparación) [VOL02-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0024] (Avanzado, Comprobación de hipótesis) [VOL02-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0025] (Nivel de examen, Optimización) [VOL02-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0026] (Nivel de examen, Demostración) [VOL02-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0027] (Reto, Diseño) [VOL02-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0028] (Reto, Estudio de parámetros) [VOL02-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 3

Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

Distinguir con rigor matemático cuándo un problema de optimización admite una estrategia voraz local, cuándo requiere programación dinámica por solapamiento de subproblemas y cuándo exige búsqueda en espacio de estados con poda (backtracking / branch and bound).

Consejo

Al optimizar espacio en la mochila 0-1 reduciendo la matriz 2D a un único vector 1D `dp[0..C]`, recorra siempre las capacidades en orden decreciente `for c in (w..=C).rev()`: si las recorre en orden creciente, estará reutilizando el mismo objeto múltiples veces (resolviendo la mochila ilimitada Unbounded Knapsack en lugar de la mochila 0-1).

Definición

Ecuación de Recurrencia de Bellman en la Mochila 0-1 y Distancia de Edición (Levenshtein): para n objetos con pesos w_i y valores v_i bajo capacidad C, el valor óptimo satisface $OPT(i, c) = \max(OPT(i-1, c),\ v_i + OPT(i-1, c-w_i))$ si $w_i \leq c$, y \$OPT(i-1, c)\$ en caso contrario, con complejidad pseudopolinómica $\Theta(nC)$ en tiempo y $\Theta(C)$ en espacio.

Autoficha

1. ¿Qué contraejemplo mínimo demuestra que el algoritmo voraz falla en el cambio de monedas con denominaciones `{1, 3, 4}`? 2. ¿Por qué recorrer las capacidades hacia atrás en `dp[c]` distingue la mochila 0-1 de la mochila ilimitada? 3. ¿Por qué la complejidad $\Theta(nC)$ de la mochila 0-1 se denomina pseudopolinómica?

3.1 Divide y vencerás y condición de elección voraz (Greedy Choice Property)

Un problema exhibe subestructura óptima cuando una solución óptima global contiene dentro de sí soluciones óptimas a sus subproblemas. Si los subproblemas son disjuntos (independientes), el paradigma de Divide y Vencerás los resuelve recursivamente y combina sus resultados (Mergesort, transformada rápida de Fourier FFT, multiplicación de enteros de Karatsuba en $O(n^{\log_2 3}) \approx O(n^{1{,}585})$). Los algoritmos voraces (greedy) dan un paso más audaz: en cada etapa toman una decisión localmente óptima sin explorar las ramas alternativas. Un algoritmo voraz es correcto única y exclusivamente cuando se demuestra la propiedad de elección voraz (existe siempre una solución óptima global que comienza con la elección voraz) junto con la subestructura óptima. Así ocurre en la selección de actividades por tiempo de finalización más temprano, en la codificación prefijo de entropía mínima de Huffman y en los árboles generadores mínimos sobre matroides (Kruskal y Prim); en cambio, en el problema de la mochila 0-1 (0-1 Knapsack) o del cambio de monedas para sistemas no canónicos (como monedas `{1, 3, 4}` para sumar `6`, donde el voraz elige `4+1+1` = 3 monedas frente al óptimo `3+3` = 2 monedas), la heurística voraz falla y es necesario recurrir a programación dinámica.

3.2 Programación dinámica (Bellman) y búsqueda exhaustiva con poda (Backtracking)

La Programación Dinámica (introducida por Richard Bellman en los años cincuenta) resuelve problemas con subestructura óptima y subproblemas solapados calculando cada estado una sola vez. Todo diseño de programación dinámica sigue cuatro pasos formales: (1) definir el espacio de estados y su significado, (2)

escribir la ecuación de recurrencia de Bellman relacionando un estado con sus predecesores en el DAG de dependencias, (3) establecer los casos base, y (4) elegir el orden de evaluación: recursivo con caché (memoización top-down) o iterativo en orden topológico (tabulación bottom-up), reduciendo a menudo la complejidad espacial de $O(n \cdot W)$ a $O(W)$ cuando la fila actual solo depende de la anterior. Cuando el problema es NP-duro y carece de estructura polinómica compacta (como satisfacibilidad booleana SAT, coloreado de grafos o las N-reinas), el Backtracking explora sistemáticamente el árbol de decisiones en profundidad aplicando predicados de factibilidad y cotas de optimalidad (branch and bound) para podar subárboles enteros antes de expandirlos:

```
def knapsack_01(weights: list[int], values: list[int], capacity: int) -> int:
    dp = [0] * (capacity + 1)
    for w, val in zip(weights, values):
        for c in range(capacity, w - 1, -1):
            if dp[c - w] + val > dp[c]:
                dp[c] = dp[c - w] + val
    return dp[capacity]

# Voraz por densidad elegiría (w=10, v=60)+(w=20, v=100) = 160; óptimo DP toma (20,100)
    +(30,120) = 220
assert knapsack_01([10, 20, 30], [60, 100, 120], 50) == 220
```

Problema resuelto

Enunciado En el algoritmo de Karatsuba para multiplicar dos enteros de n dígitos dividiendo cada operando en dos mitades, se sustituyen las 4 multiplicaciones clásicas de tamaño $n/2$ por solo 3 multiplicaciones recursivas. Compare el número de multiplicaciones base en una profundidad de $k = 4$ niveles ($n = 16$ bloques) entre el método cuadrático clásico (\$4^4\$) y Karatsuba (\$3^4\$), calculando el ahorro neto de productos elementales.

Datos

- $k = 4$ — Niveles de recursión (para $n = 2^4 = 16$)
- $b_{classic} = 4$ — Subproblemas por nivel en método clásico
- $b_{karat} = 3$ — Subproblemas por nivel en Karatsuba

Incógnita Ahorro de multiplicaciones base tras 4 niveles de recursión

Modelo físico

Árbol de recursión de Divide y Vencerás (4^k frente a 3^k)

$$saved = M_classic - 3^4$$

Hipótesis

- Cada nivel divide exactamente a la mitad el ancho de palabra de los operandos

Desarrollo

1. Calculamos los productos en las hojas del algoritmo cuadrático clásico 4^4.

$$M_classic = 4^4$$

$$(4)^4$$

2. Restamos los productos en las hojas de Karatsuba $3^4 = 81$.

$$saved = M_classic - 3^4$$

$$(256 - 81)$$

Resultado

175

Interpretación En solo 4 niveles de subdivisión Karatsuba reduce de 256 a 81 los productos base (ahorro de 175); para operandos criptográficos de miles de bits la ventaja asintótica $O(n^{1}.585)$ es decisiva.

Comprobación de los pasos

- Paso 1: 256 — reproducido
- Paso 2: 175 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Para operandos pequeños (p. ej. menos de 32 palabras de 64 bits), la sobrecarga de sumas y restas hace más rápido el producto escolar en ensamblador.

Errores frecuentes

- Aplicar programación dinámica cuando los subproblemas nunca se solapan (donde Divide y Vencerás no necesita tabla de memoización).

Ejercicios

1 [PRB-0031] (Básico, Conceptual) [VOL02-C03-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0032] (Básico, Numérico) [VOL02-C03-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0033] (Intermedio, Interpretación) [VOL02-C03-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0034] (Intermedio, Aplicación) [VOL02-C03-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0035] (Intermedio, Identificación de errores) [VOL02-C03-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0036] (Avanzado, Desarrollo por pasos) [VOL02-C03-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0037] (Avanzado, Comparación) [VOL02-C03-E07] (Nivel 4– Ingeniería (Refactoriza-

ción modular y contratos)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0038] (Avanzado, Comprobación de hipótesis) [VOL02-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0039] (Nivel de examen, Optimización) [VOL02-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0040] (Nivel de examen, Demostración) [VOL02-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0041] (Reto, Diseño) [VOL02-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0042] (Reto, Estudio de parámetros) [VOL02-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 4

Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

Comprender en profundidad los algoritmos sobre grafos y las dos grandes familias de índices asociativos —árboles equilibrados y tablas hash— que sustentan desde los enrutadores de red y compiladores hasta los motores de bases de datos y las colecciones estándar de todos los lenguajes.

Consejo

Si una tabla hash recibe claves controladas por usuarios externos (parámetros HTTP, cabeceras JSON) y utiliza una función hash no criptográfica predecible, un atacante puede enviar miles de claves que colisionen en la misma cubeta y degradar la complejidad de $O(1)$ a $O(n^2)$ (denegación de servicio HashDoS): por eso Rust usa un hasher con semilla aleatoria por defecto.

Definición

Factor de Carga (α) en Tablas Hash y Altura de un Árbol B+: en una tabla hash con n elementos y m ranuras, el factor de carga es $\alpha = n/m$. En direccionamiento abierto con sondeo lineal bajo dispersión uniforme, el número esperado de sondeos en una búsqueda exitosa es $\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$. En un árbol B+ de grado mínimo B, la altura satisface $h \leq \lfloor \log_B((n+1)/2) \rfloor + 1$.

Autoficha

1. ¿Por qué el algoritmo de Dijkstra falla si el grafo contiene una arista de peso negativo? 2. ¿Por qué `BTreeMap` y los motores SQL usan nodos de árbol B/B+ con múltiples claves por nodo en lugar de árboles binarios Rojo-Negro? 3. ¿En qué consiste un ataque de colisión HashDoS y cómo lo mitiga SipHash con semilla aleatoria?

4.1 BFS, DFS, componentes conexas, Dijkstra, Bellman-Ford, Kruskal y Prim

Sobre un grafo $G = (V, E)$ representado mediante listas de adyacencia (espacio $\Theta(|V|+|E|)$), la búsqueda en anchura (BFS, gobernada por una cola FIFO) explora el grafo por capas y calcula distancias mínimas en número de aristas en tiempo $\Theta(|V|+|E|)$. La búsqueda en profundidad (DFS, recursiva o con pila explícita) clasifica las aristas en árbol, avance, retroceso (cuya presencia caracteriza exactamente los ciclos en un grafo dirigido) y cruce, permitiendo calcular en tiempo lineal $\Theta(|V|+|E|)$ la ordenación topológica de un DAG y las componentes fuertemente conexas (algoritmo de Tarjan o Kosaraju). Cuando las aristas tienen pesos no negativos $w(u, v) \geq 0$, el algoritmo de Dijkstra relaja distancias extrayendo el vértice de menor distancia tentativa desde una cola de prioridad (montículo binario), alcanzando $O((|V| + |E|) \log |V|)$. Si existen pesos negativos, la hipótesis voraz de Dijkstra falla (un camino ya cerrado podría mejorarse más tarde a través de una arista negativa), y debe emplearse Bellman-Ford, que relaja todas las aristas $|V|-1$ veces en $O(|V| \cdot |E|)$ y detecta ciclos negativos alcanzables en la iteración $|V|$.

4.2 Árboles equilibrados (AVL, Rojo-Negro, B+ Trees) y tablas hash de alto rendimiento

Para implementar diccionarios dinámicos existen dos compromisos fundamentales. Los árboles binarios de búsqueda autoequilibrados (AVL con factor de equilibrio $|h_L - h_R| \leq 1$, y Rojo-Negro con altura acotada por $2\log_2(n+1)$) garantizan búsqueda, inserción, borrado y consultas por rango en $\Theta(\log n)$ en el peor caso, pero cada salto de puntero suele provocar un fallo de caché. Por ello, tanto `std::collections::BTreeMap` en Rust como los índices de bases de datos (PostgreSQL, SQLite) utilizan árboles B y B+ de alto factor de ramificación B: cada nodo agrupa entre $B-1$ y $2B-1$ claves contiguas en memoria o páginas de disco de 4–16 KiB, reduciendo la altura del árbol a $\log_B n$ (apenas 3 o 4 niveles para cientos de millones de registros). Las tablas hash proyectan cada clave mediante una función de dispersión $h(k)$ a un índice de cubeta (bucket), ofreciendo acceso esperado $O(1)$. Frente al encadenamiento separado con listas enlazadas, las implementaciones modernas de producción (como `HashMap` en Rust basado en SwissTable de Google con instrucciones SIMD y SipHash-1-3 resistente a ataques HashDoS, o `dict` compacto en Python) emplean direccionamiento abierto (open addressing) en un array contiguo con factor de carga $\alpha = n/m \leq 0{,}875$. Verifiquemos el algoritmo de Dijkstra en Python:

```
import heapq

def dijkstra(adj: dict[int, list[tuple[int, int]]], src: int) -> dict[int, int]:
    dist = {src: 0}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, 10**18):
            continue
        for v, w in adj.get(u, []):
            if d + w < dist.get(v, 10**18):
                dist[v] = d + w
                heapq.heappush(pq, (d + w, v))
    return dist

g = {0: [(1, 4), (2, 1)], 2: [(1, 2), (3, 5)], 1: [(3, 1)], 3: []}
assert dijkstra(g, 0) == {0: 0, 2: 1, 1: 3, 3: 4}
```

Problema resuelto

Enunciado En una tabla hash con direccionamiento abierto y sondeo lineal que se mantiene con un factor de carga $\alpha = 0{,}75$, calcule el número esperado de sondeos en una búsqueda sin éxito (inserción de clave nueva) según la fórmula de Knuth $E_{\text{miss}} = \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$.

Datos

- $\alpha = 0{,}75$ — Factor de carga n / m de la tabla hash

Incógnita Número medio esperado de sondeos E_miss en una búsqueda sin éxito / inserción

Modelo físico

Fórmula de Knuth para agrupamiento primario (primary clustering) en sondeo lineal

$$E_miss = 0{,}5 * (1 + inv_sq)$$

Hipótesis

- Función hash uniforme e independiente sobre m ranuras

Desarrollo

1. Calculamos el inverso del cuadrado de la fracción libre $(1 - 0{,}75)^2 = 0{,}0625$.

$$inv_sq = 1/((1 - 0{,}75)^2)$$

$$\frac{1}{((1-0{,}75))^2}$$

2. Aplicamos la media 0.5 * (1 + inv$_s q$) para obtener el número esperado de sondeos.

$$E_m iss = 0{,}5 * (1 + inv_s q)$$

$$(0{,}5 \cdot (1 + 16))$$

Resultado

$$8{,}5$$

Interpretación Al 75 % de ocupación, una inserción por sondeo lineal clásico explora 8.5 ranuras en promedio (y al 90 % subiría a 50.5 ranuras); de ahí el uso de Robin Hood hashing o metadatos SIMD (SwissTable).

Comprobación de los pasos

- Paso 1: 16 — reproducido
- Paso 2: 8,5 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- El resultado asume independencia total de la función hash; funciones débiles sufren agrupamiento aún mayor.

Errores frecuentes

- Permitir que una tabla de direccionamiento abierto supere alpha > 0.85 sin redimensionar.

Ejercicios

1 [PRB-0045] (Básico, Conceptual) [VOL02-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0046] (Básico, Numérico) [VOL02-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0047] (Intermedio, Interpretación) [VOL02-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0048] (Intermedio, Aplicación) [VOL02-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0049] (Intermedio, Identificación de errores) [VOL02-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre algoritmos de grafos, árboles B+ y tablas hash:

analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0050] (Avanzado, Desarrollo por pasos) [VOL02-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0051] (Avanzado, Comparación) [VOL02-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0052] (Avanzado, Comprobación de hipótesis) [VOL02-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0053] (Nivel de examen, Optimización) [VOL02-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0054] (Nivel de examen, Demostración) [VOL02-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0055] (Reto, Diseño) [VOL02-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0056] (Reto, Estudio de parámetros) [VOL02-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 5

Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

Dominar técnicas avanzadas indispensables en sistemas modernos: estructuras persistentes que conservan versiones históricas sin clonar toda la memoria, filtros de Bloom para evitar accesos innecesarios a disco y el modelo Work-Span para analizar algoritmos paralelos.

Consejo

En un algoritmo paralelo de divide y vencerás (como Mergesort o suma en árbol), corte la generación de tareas paralelas por debajo de un umbral de granularidad (por ejemplo, bloques de 4096 elementos) y resuelva las hojas secuencialmente para que el coste de planificación no domine a T_1.

Definición

Teorema de Brent-Graham en el Modelo Work-Span y Paralelismo Medio: dado un cómputo paralelo libre de bloqueos con trabajo total T_1 y camino crítico (span) T_∞, el tiempo de ejecución en P procesadores bajo planificación voraz satisface $\max(T_1/P, T_\infty) \leq T_P \leq T_1/P + T_\infty$. El paralelismo medio es $\overline{P} = T_1/T_\infty$.

Autoficha

1. ¿Por qué la compartición estructural (path copying) solo es segura cuando los nodos compartidos son inmutables? 2. ¿Por qué un filtro de Bloom nunca produce falsos negativos pero sí puede producir falsos positivos, y por qué no admite borrado directo de bits? 3. ¿Qué mide el cociente T_1/T_∞ en el modelo Work-Span?

5.1 Estructuras persistentes (path copying) y filtros probabilísticos (Bloom Filter)

Una estructura de datos es persistente cuando cualquier operación de actualización produce una nueva versión lógicamente independiente conservando intacta la versión anterior. En lugar de copiar los n elementos en $\Theta(n)$, las estructuras persistentes basadas en árboles (empleadas en Git, Clojure, Scala, Redux/Immer y motores MVCC) aplican copia de camino (path copying) y compartición estructural (structural sharing): al modificar una hoja de un árbol de altura $h = O(\log n)$, se asignan únicamente los $O(\log n)$ nodos del camino desde la raíz hasta la hoja modificada, mientras que todos los subárboles no modificados se comparten de forma segura porque son inmutables. En sistemas de almacenamiento (como motores LSM-tree en RocksDB, Cassandra o LevelDB), antes de buscar una clave en múltiples archivos SSTable de disco se consulta en RAM un Filtro de Bloom (Burton Bloom, 1970): un vector de m bits inicializado a 0 junto con k funciones hash independientes. Al insertar x, se activan los bits $h_1(x), \ldots, h_k(x)$. Si al consultar y alguno de los k bits vale 0, se garantiza con certeza absoluta (0 % de falsos negativos) que y no pertenece al conjunto; si todos valen 1, puede tratarse de un falso positivo con probabilidad aproximada $(1 - e^{-kn/m})^k$, que se minimiza eligiendo $k = \frac{m}{n} \ln 2$.

5.2 Algoritmos paralelos: modelo Work-Span, ley de Brent y reducciones paralelas

Para analizar algoritmos paralelos con independencia del número concreto de núcleos físicos P, el modelo de grafo de tareas (Work-Span model, usado en Cilk, Rayon en Rust y ForkJoinPool en Java) define dos métricas intrínsecas del cómputo DAG: 1. Trabajo (T_1, Work): el tiempo total de ejecutar todas las operaciones en un único procesador. 2. Longitud de camino crítico (T_∞, Span o Depth): el tiempo requerido en un computador ideal con infinitos procesadores, determinado por la cadena más larga de dependencias secuenciales. Todo planificador válido satisface las dos cotas inferiores fundamentales: $T_P \geq T_1/P$ (cota de trabajo) y $T_P \geq T_\infty$ (cota de camino crítico). Además, un planificador voraz o por robo de trabajo (work-stealing, como el de Rayon o Tokio) garantiza la cota superior de Brent-Graham: $T_P \leq T_1/P + T_\infty$. El cociente $\overline{P} = T_1/T_\infty$ se denomina paralelismo medio del algoritmo: cuando $\overline{P} \gg P$, el algoritmo alcanza aceleración lineal casi perfecta $T_P \approx T_1/P$.

Problema resuelto

Enunciado Un algoritmo paralelo de reducción y filtrado tiene un trabajo total $T_1 = 9600$ ms y un camino crítico (span) $T_\infty = 40$ ms. Calcule la cota superior garantizada de tiempo de ejecución $T_P = T_1/P + T_\infty$ en $P = 16$ procesadores bajo un planificador voraz.
Datos

- $T_1 = 9600$ — Trabajo secuencial total (ms)
- $T_{inf} = 40$ — Camino crítico o span (ms)
- $P = 16$ — Número de procesadores

Incógnita Cota superior de tiempo de ejecución T_P en milisegundos
Modelo físico
Desigualdad de Brent-Graham en el modelo Work-Span (T_P <= T_1 / P + T_inf)

$$T_{P,\text{máx}} = t_{extwork} + T_\infty$$

Hipótesis

- Planificador voraz sin sobrecarga significativa de robo de tareas

Desarrollo

1. Dividimos el trabajo total T_1 entre los P = 16 procesadores.

$$t_{extwork} = T_1/P$$

$$\frac{9600}{16}$$

2. Sumamos el span $T_inf = 40$ ms para obtener la cota superior garantizada.

$$T_{P,\text{máx}} = t_{extwork} + T_\infty$$

$$(600 + 40)$$

Resultado

$$640$$

Interpretación Dado que el paralelismo medio $T_1/T_inf = 240$ supera ampliamente a P = 16, el tiempo queda acotado entre 600 ms y 640 ms, logrando una aceleración efectiva de 9600 / 640 = 15x.
Comprobación de los pasos

- Paso 1: 600 — reproducido
- Paso 2: 640 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- La cota no contabiliza la contención del ancho de banda de memoria DRAM cuando los 16 núcleos saturan el bus.

Errores frecuentes

- Intentar borrar un elemento de un filtro de Bloom estándar poniendo sus k bits a 0, lo cual corrompería otros elementos que compartan esos bits.

Ejercicios

1 [PRB-0059] (Básico, Conceptual) [VOL02-C05-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0060] (Básico, Numérico) [VOL02-C05-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0061] (Intermedio, Interpretación) [VOL02-C05-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0062] (Intermedio, Aplicación) [VOL02-C05-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0063] (Intermedio, Identificación de errores) [VOL02-C05-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0064] (Avanzado, Desarrollo por pasos) [VOL02-C05-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0065] (Avanzado, Comparación) [VOL02-C05-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0066] (Avanzado, Comprobación de hipótesis) [VOL02-C05-E08] (Nivel 4– Ingeniería (Validación de precondiciones y bordes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0067] (Nivel de examen, Optimización) [VOL02-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0068] (Nivel de examen, Demostración) [VOL02-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0069] (Reto, Diseño) [VOL02-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0070] (Reto, Estudio de parámetros) [VOL02-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Parte II

PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos

Capítulo 6

Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

Explicar cómo las líneas de caché de 64 bytes, el protocolo de coherencia entre núcleos, las reglas de alineamiento del ABI y el proceso de enlazado determinan el rendimiento real y la interoperabilidad binaria de los programas.

Consejo

En C, ordene los miembros de un `struct` de mayor a menor requisito de alineamiento (primero punteros y `uint64_t` de 8 bytes, luego `uint32_t` de 4, `uint16_t` de 2 y finalmente `uint8_t`/`bool` de 1 byte) para eliminar el padding interno y aumentar el número de registros que caben por línea de caché.

Definición

Regla de Alineamiento de Estructuras (ABI C) y Compartición Falsa (False Sharing): en un `struct` con diseño C, cada campo con alineamiento a_i se sitúa en el menor desplazamiento `offset` tal que offset mód $a_i = 0$, y el tamaño total del `struct` se redondea al múltiplo superior de $a_{\text{máx}} = \text{máx}_i\, a_i$. La compartición falsa ocurre cuando múltiples núcleos escriben en variables disjuntas que residen en la misma línea física de caché.

Autoficha

1. ¿Por qué `struct { uint8_t a; uint64_t b; uint8_t c; }` ocupa 24 bytes en C de 64 bits pero solo 16 bytes si se reordena como `{ uint64_t b; uint8_t a; uint8_t c; }`? 2. ¿Qué provoca el fenómeno de false sharing en el protocolo MESI y cómo se corrige? 3. ¿Por qué Rust distingue entre su layout optimizado por defecto y `#[repr(C)]`?

6.1 Líneas de caché, coherencia MESI, false sharing y diseño orientado a datos

Las cachés del procesador no transfieren bytes individuales desde la RAM, sino bloques alineados de tamaño fijo denominados líneas de caché (cache lines, de 64 bytes en x86-64 y la mayoría de procesadores ARM64, o 128 bytes en los núcleos de alto rendimiento Apple M-series). En un procesador multinúcleo, cada núcleo posee su propia caché L1 privada; para evitar lecturas incoherentes, el hardware implementa protocolos de invalidación de directorio o bus como MESI (Modified, Exclusive, Shared, Invalid): para que un núcleo modifique un byte de una línea, debe adquirir la línea completa en estado Exclusive/Modified e invalidar las copias en las cachés L1 de todos los demás núcleos. De aquí surge una patología crítica de rendimiento concurrente: la compartición falsa (false sharing). Si dos hilos ejecutándose en núcleos distintos actualizan intensivamente dos contadores atómicos independientes (`counter_a` y `counter_b`) que por estar declarados contiguos en una estructura caen dentro de la misma línea de caché de 64 bytes, ambos núcleos se invalidarán mutuamente la línea en cada escritura (cache line ping-pong), degradando el rendimiento en un orden de magnitud pese a no compartir ningún dato lógico. La solución consiste en acumular en una variable local de registro y volcar al final, o alinear cada contador a su propia línea de caché (`#[repr(align(64))]` en Rust o `alignas(64)` en C11/C++11).

6.2 Alineamiento en memoria, ABI de plataforma y enlazadores (ELF/Mach-O)

Los accesos a memoria de un tipo primitivo de tamaño k bytes son atómicos y eficientes en hardware cuando su dirección es múltiplo de su alineamiento natural k. Por ello, el ABI de C (como el System V AMD64 ABI o el AAPCS64 de ARM) inserta bytes de relleno (padding) entre campos de un `struct` y al final del mismo para que su tamaño total sea múltiplo del alineamiento máximo de sus miembros. Mientras que en C el compilador está obligado a respetar el orden textual de declaración de los campos, en Rust el diseño por defecto `#[repr(Rust)]` permite al compilador reordenar automáticamente los campos en memoria para minimizar el padding sin cambiar el código fuente (utilizándose `#[repr(C)]` solo cuando se cruza una frontera FFI). Cuando el compilador traduce una unidad de traducción, emite un archivo objeto relocalizable (formato ELF en Linux, Mach-O en macOS, COFF/PE en Windows) con tablas de símbolos definidos e indefinidos y entradas de relocalización. El enlazador (linker: `lld`, `ld`, `mold`) resuelve las referencias cruzadas entre módulos, fusiona secciones y produce bien un ejecutable enlazado estáticamente o bien un binario dinámico que delega en el cargador del sistema (`ld.so` / `dyld`) la resolución de bibliotecas compartidas (`.so` / `.dylib` / `.dll`) a través de la Global Offset Table (GOT) y Procedure Linkage Table (PLT).

Problema resuelto

Enunciado Calcule el ahorro total en kilobytes (1 KiB = 1024 bytes) de un vector en memoria de $N = 65536$ registros cuando se reordena en C una estructura mal alineada de 24 bytes (`uint8_t`, 7 bytes de padding, `uint64_t`, `uint8_t`, 7 bytes de padding) a su disposición óptima de 16 bytes (`uint64_t`, `uint8_t`, `uint8_t`, 6 bytes de padding final).
Datos

- $N = 65536$ — Número de estructuras en el vector
- $S_{bad} = 24$ — Tamaño en bytes con orden subóptimo
- $S_{opt} = 16$ — Tamaño en bytes ordenando de mayor a menor alineamiento

Incógnita Ahorro de memoria en KiB para el vector de 65536 elementos
Modelo físico
Cálculo de huella de memoria de arrays contiguos con alineamiento ABI de 8 bytes

$$saved_kib = (N * delta_s)/1024$$

Hipótesis

- Arquitectura de 64 bits donde alignof(uint64$_t$) == 8

Desarrollo

1. Calculamos los bytes ahorrados por cada registro individual al eliminar un bloque de padding.

$$delta_s = S_bad - S_opt$$

$$(24 - 16)$$

2. Multiplicamos por los 65536 registros y dividimos entre 1024 bytes/KiB.

$$saved_kib = (N * delta_s)/1024$$

$$\frac{(65536 \cdot 8)}{1024}$$

Resultado

512

Interpretación Simplemente reordenando dos líneas de declaración en C se ahorran 512 KiB (un 33 % de la memoria) y ahora caben 4 estructuras por línea de caché de 64 B en lugar de 2.66.
Comprobación de los pasos

- Paso 1: 8 — reproducido
- Paso 2: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Si la estructura define un protocolo binario de red o hardware fijo, no debe reordenarse sin actualizar la especificación del protocolo.

Errores frecuentes

- Abusar de `__attribute__((packed))` o `#[repr(packed)]` en estructuras de uso intensivo en CPU, generando accesos desalineados costosos o UB al tomar referencias.

Ejercicios

1 [PRB-0073] (Básico, Conceptual) [VOL02-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0074] (Básico, Numérico) [VOL02-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0075] (Intermedio, Interpretación) [VOL02-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0076] (Intermedio, Aplicación) [VOL02-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(24 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0077] (Intermedio, Identificación de errores) [VOL02-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0078] (Avanzado, Desarrollo por pasos) [VOL02-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0079] (Avanzado, Comparación) [VOL02-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica carac-

terística mediante la expresión reproducible `24 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0080] (Avanzado, Comprobación de hipótesis) [VOL02-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0081] (Nivel de examen, Optimización) [VOL02-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0082] (Nivel de examen, Demostración) [VOL02-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0083] (Reto, Diseño) [VOL02-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0084] (Reto, Estudio de parámetros) [VOL02-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 7

Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

Desmitificar cómo funciona por dentro un compilador industrial (`rustc`, `clang`, `gcc`, `ghc`, `v8`), recorriendo las etapas de la figura ?? para que el lector pueda tanto diagnosticar qué código genera su compilador como construir su propio lenguaje, DSL o herramienta de análisis estático.

Consejo

Cuando escriba un parser para expresiones aritméticas o booleanas con múltiples niveles de precedencia, utilice un parser de Pratt con potencias de enlace izquierda/derecha `(l_bp, r_bp)` en lugar de crear una función recursiva distinta por cada nivel de precedencia: el código se reduce a una tercera parte y permite añadir nuevos operadores en una sola línea de tabla.

Definición

Forma de Asignación Única Estática (SSA) y Grafo de Interferencia de Registros: una representación intermedia está en forma SSA si cada variable virtual posee exactamente un punto de definición estático y toda definición domina todos sus usos (o se combina mediante nodos ϕ en la frontera de dominancia). Dos variables interfieren en la asignación de registros si sus intervalos de vida (liveness) se solapan en algún punto del CFG, impidiendo compartir el mismo registro físico.

Autoficha

1. ¿Por qué los compiladores industriales como `rustc` y `clang` prefieren descenso recursivo + Pratt frente a generadores LR automáticos? 2. ¿Para qué sirve el nodo $\phi(x_1, x_2)$ en la representación intermedia SSA cuando convergen dos ramas de un `if-else`? 3. ¿Cómo se modela la asignación de registros como un problema de k-coloreado sobre el grafo de interferencia?

7.1 Front-end del compilador: análisis léxico, sintáctico (descendente recursivo y Pratt) y semántico

Un compilador moderno se divide en tres grandes etapas desacopladas (Front-End, Middle-End y Back-End). El Front-End comienza con el analizador léxico (lexer o scanner), que agrupa caracteres UTF-8 mediante autómatas finitos deterministas en una secuencia de tokens con información de posición (span). A continuación, el analizador sintáctico (parser) comprueba la gramática libre de contexto y construye el Árbol de Sintaxis Abstracta (AST). Mientras que los generadores clásicos (Yacc/Bison, ANTLR) construyen tablas LR o LL, los compiladores de producción actuales (`rustc`, `clang`, `go`, `tsc`) utilizan mayoritariamente descenso recursivo escrito a mano combinado con parsing de Pratt (Top-Down Operator Precedence, Vaughan Pratt 1973) para expresiones con niveles de precedencia y asociatividad, ya que facilita mensajes de diagnóstico precisos y recuperación ante errores sintácticos. Sobre el AST, el análisis semántico resuelve nombres en tablas de símbolos con ámbito léxico y ejecuta la comprobación e inferencia de tipos (mediante unificación de restricciones al estilo Hindley-Milner Damas-Milner o comprobación bidireccional), reduciendo el lenguaje de superficie a una representación tipada de alto nivel (como HIR y MIR en `rustc`, donde se ejecuta el borrow checker).

7.2 Representación intermedia SSA, optimizaciones e instrucción/asignación de registros

El Middle-End traduce el programa a un Grafo de Flujo de Control (CFG) de bloques básicos en forma SSA (Static Single Assignment, Cytron et al. 1991, base de LLVM IR, GCC GIMPLE y Go SSA): cada variable temporal se asigna exactamente una vez en el texto del programa, y cuando dos ramas de un condicional convergen en un bloque de unión se inserta una función $\phi(x_1, x_2)$ que selecciona el valor según el bloque predecesor. La forma SSA hace explícitas las cadenas definición-uso (def-use chains), simplificando drásticamente optimizaciones como propagación de constantes, eliminación de código muerto (DCE), numeración global de valores (GVN), movimiento de código invariante de bucle (LICM) e inlining. Finalmente, el Back-End realiza la selección de instrucciones para la ISA destino (x86-64, ARM64, RISC-V, WebAssembly), ejecuta la asignación de registros (modelada como coloreado de grafos de interferencia por Chaitin-Briggs o mediante barrido lineal Linear Scan en compiladores JIT, volcando a la pila −spilling− las variables que exceden los registros físicos disponibles) y planifica las instrucciones para maximizar el paralelismo a nivel de instrucción.

Problema resuelto

Enunciado En un bloque básico antes de la optimización se evalúa en un bucle de $N = 1000$ iteraciones una expresión con una subexpresión invariante de bucle de coste $c_{\text{inv}} = 6$ ciclos y una parte dependiente del índice de coste $c_{\text{var}} = 3$ ciclos por iteración. Calcule el ahorro total de ciclos al aplicar el pase de Movimiento de Código Invariante de Bucle (LICM), que evalúa la subexpresión invariante una única vez fuera del bucle.

Datos

- $N = 1000$ — Iteraciones del bucle
- $c_{inv} = 6$ — Ciclos de la subexpresión invariante
- $c_{var} = 3$ — Ciclos de la parte dependiente del índice

Incógnita Ahorro neto de ciclos tras aplicar el pase LICM

Modelo físico

Modelo de coste de bloque básico bajo Loop-Invariant Code Motion (LICM)

$$saved = C_orig - (c_inv + N * c_var)$$

Hipótesis

- La subexpresión invariante es pura, no lanza trampas de hardware (como división por cero) y sus operandos no sufren aliasing en el bucle

Desarrollo

1. Calculamos el coste sin optimizar ejecutando ambas partes en las 1000 iteraciones.

$$C_orig = N * (c_inv + c_var)$$

$$(1000 \cdot (6 + 3))$$

2. Restamos el coste tras elevar la expresión invariante al preheader del bucle (6 + 3000).

$$saved = C_orig - (c_inv + N * c_var)$$

$$(9000 - 3006)$$

Resultado

$$5994$$

Interpretación El pase LICM ahorra 5994 ciclos (dos tercios del tiempo del bucle); en C este pase puede bloquearse si el compilador no puede descartar que un puntero de escritura apunte al mismo objeto leído (problema que se resuelve con `restrict` en C o con las garantías de `&` y `&mut` en Rust).
Comprobación de los pasos

- Paso 1: 9000 — reproducido
- Paso 2: 5994 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Elevar demasiadas expresiones invariantes simultáneamente puede aumentar la presión de registros y provocar spilling a la pila.

Errores frecuentes

- Suponer que el compilador puede sacar del bucle una llamada a función externa con posibles efectos laterales o lecturas a través de punteros con posible aliasing.

Ejercicios

1 [PRB-0087] (Básico, Conceptual) [VOL02-C07-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0088] (Básico, Numérico) [VOL02-C07-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0089] (Intermedio, Interpretación) [VOL02-C07-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0090] (Intermedio, Aplicación) [VOL02-C07-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(26 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0091] (Intermedio, Identificación de errores) [VOL02-C07-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0092] (Avanzado, Desarrollo por pasos) [VOL02-C07-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0093] (Avanzado, Comparación) [VOL02-C07-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0094] (Avanzado, Comprobación de hipótesis) [VOL02-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0095] (Nivel de examen, Optimización) [VOL02-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0096] (Nivel de examen, Demostración) [VOL02-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0097] (Reto, Diseño) [VOL02-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0098] (Reto, Estudio de parámetros) [VOL02-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 8

Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

Comprender las abstracciones fundamentales que el núcleo del sistema operativo (Linux, XNU/macOS, Windows NT) ofrece a todo programa: aislamiento de procesos mediante memoria virtual, coste real de las llamadas al sistema y mecanismos de entrada/salida de alta concurrencia.

Consejo

Minimice el número de llamadas al sistema (`read`/`write`) en bucles críticos utilizando buffers en espacio de usuario (`BufReader`/`BufWriter` en Rust, `BufferedInputStream` en Java, `stdio` en C): escribir 1 MiB byte a byte mediante un millón de llamadas `write(fd, &b, 1)` es cientos de veces más lento que agrupar en bloques de 8 KiB o 64 KiB.

Definición

Traducción de Direcciones Virtuales y Multiplexación de Eventos $O(1)$: en un sistema con páginas de 2^p bytes, los p bits menos significativos de una dirección virtual forman el desplazamiento dentro de la página (page offset, que no se traduce) y los bits superiores forman el número de página virtual (VPN) traducido por la TLB/tabla de páginas al número de marco físico (PFN). Primitivas como `epoll` y `kqueue` notifican únicamente los descriptores listos sin escanear el conjunto total de descriptores registrados.

Autoficha

1. ¿Qué recursos comparten dos hilos de un mismo proceso y qué recursos son privados de cada hilo? 2. ¿Qué función cumple la TLB en la traducción de direcciones virtuales y por qué un cambio de proceso encarece más el contexto que un cambio de hilo del mismo proceso? 3. ¿Por qué `epoll`/`kqueue` escalan a decenas de miles de conexiones donde `select`/`poll` se degradan linealmente?

8.1 Modo usuario vs modo kernel, llamadas al sistema, procesos, hilos y planificación

El procesador impone aislamiento de hardware mediante al menos dos niveles de privilegio: modo usuario (Ring 3 en x86-64, EL0 en ARM64) y modo supervisor o kernel (Ring 0 / EL1). El código de una aplicación se ejecuta en modo usuario y no puede acceder directamente a dispositivos físicos ni modificar tablas de páginas; para solicitar servicios al núcleo (abrir un archivo, reservar páginas de memoria con `mmap`, enviar paquetes por un socket) ejecuta una instrucción de trampa síncrona (`syscall` en x86-64, `svc #0` en ARM64) que salva los registros de usuario, conmuta a la pila del kernel, valida los argumentos y ejecuta el manejador privilegiado. Un proceso es la unidad de aislamiento de recursos: posee su propio espacio de direcciones virtuales, tabla de descriptores de archivo y credenciales de seguridad. Un hilo (thread) es la unidad de planificación de CPU: varios hilos dentro de un mismo proceso comparten el espacio de direcciones virtuales (código, datos globales y heap), pero cada uno posee su propio contador de programa, banco de registros y pila de llamadas independiente.

8.2 Memoria virtual, tablas de páginas, TLB, sistemas de archivos y E/S no bloqueante

La Memoria Virtual, implementada conjuntamente por la Unidad de Gestión de Memoria (MMU) del procesador y el sistema operativo, divide el espacio de direcciones en páginas de tamaño fijo (típicamente 4 KiB en x86-64 o 16 KiB en Apple Silicon ARM64). La MMU traduce cada dirección virtual en dirección física recorriendo una tabla de páginas multinivel (4 niveles en x86-64 de 48 bits) y almacenando las traducciones recientes en una caché hardware asociativa llamada TLB (Translation Lookaside Buffer). Cuando un proceso accede a una página no mapeada en RAM, la MMU dispara una excepción de fallo de página (page fault) para que el kernel asigne un marco físico bajo demanda (demand paging), cargue datos desde un archivo mapeado (`mmap`) o aplique copia en escritura (Copy-on-Write tras `fork`). En el subsistema de Entrada/Salida, bloquear un hilo por cada socket resulta inviable a gran escala; por ello los runtimes de red modernos (Tokio en Rust, Node.js/libuv, Netty en Java, Go netpoller) configuran los descriptores en modo no bloqueante (`O_NONBLOCK`) y delegan la espera multiplexada en primitivas de notificación de eventos de complejidad $O(\text{eventos activos})$: `epoll` en Linux, `kqueue` en macOS/BSD e `io_uring` (colas en anillo compartidas entre usuario y kernel sin llamadas al sistema por operación) en Linux moderno.

Problema resuelto

Enunciado En una arquitectura con páginas de 4 KiB = 4096 bytes (2^{12} bytes), dada la dirección virtual decimal $V = 20500$, determine el número de página virtual VPN $= \lfloor V/4096 \rfloor$, el desplazamiento dentro de la página offset $= V - \text{VPN} \cdot 4096$, y calcule la dirección física resultante si la tabla de páginas asigna dicho VPN al marco físico PFN = 12.
Datos

- $V = 20500$ — Dirección virtual en bytes
- $P_{size} = 4096$ — Tamaño de página en bytes (4 KiB)
- $PFN = 12$ — Número de marco físico asignado a VPN = 5

Incógnita Dirección física resultante en bytes tras la traducción MMU
Modelo físico
*Traducción de dirección virtual paginada (Physical = PFN * PageSize + Offset)*

$$P_addr = PFN * P_size + offset$$

Hipótesis

- Tamaño de página uniforme de 4096 bytes con VPN = floor(20500 / 4096) = 5

Desarrollo

1. Calculamos el desplazamiento (offset) dentro de la página 5 (5 * 4096 = 20480).

$$offset = V - 5 * P_size$$

$$(20500 - (5 \cdot 4096))$$

2. Sumamos el desplazamiento a la dirección base del marco físico PFN = 12.

$$P_addr = PFN * P_size + offset$$

$$((12 \cdot 4096) + 20)$$

Resultado

$$49172$$

Interpretación La dirección virtual 20500 corresponde a la página virtual 5 con desplazamiento de 20 bytes, que la MMU traduce al marco físico 12 en la dirección física 49152 + 20 = 49172.
Comprobación de los pasos

- Paso 1: 20 — reproducido
- Paso 2: 49172 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Para cargas de memoria de decenas de gigabytes, el uso de Huge Pages (2 MiB) reduce los fallos de TLB.

Errores frecuentes

- Confundir el tamaño de la dirección virtual con la cantidad de RAM física instalada en el equipo.

Ejercicios

1 [PRB-0101] (Básico, Conceptual) [VOL02-C08-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0102] (Básico, Numérico) [VOL02-C08-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0103] (Intermedio, Interpretación) [VOL02-C08-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0104] (Intermedio, Aplicación) [VOL02-C08-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(30 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0105] (Intermedio, Identificación de errores) [VOL02-C08-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0106] (Avanzado, Desarrollo por pasos) [VOL02-C08-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0107] (Avanzado, Comparación) [VOL02-C08-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0108] (Avanzado, Comprobación de hipótesis) [VOL02-C08-E08] (Nivel 4– Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas operativos, llamadas al sistema, memoria

virtual paginada y epoll/kqueue: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0109] (Nivel de examen, Optimización) [VOL02-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0110] (Nivel de examen, Demostración) [VOL02-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0111] (Reto, Diseño) [VOL02-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0112] (Reto, Estudio de parámetros) [VOL02-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Apéndice A

Notación

Símbolo	Significado	Unidades
$MESI$	Estados Modified, Exclusive, Shared, Invalid de coherencia de caché	adimensional
$OPT(i, c)$	Valor óptimo de la subestructura en programación dinámica	adimensional
PFN	Physical Frame Number (número de marco físico)	adimensional
T_1	Trabajo total secuencial (Work) del grafo de tareas	s
T_∞	Camino crítico (Span) con infinitos procesadores	s
VPN	Virtual Page Number (número de página virtual)	adimensional
$\Phi(D_i)$	Función de potencial de la estructura de datos en el paso i	adimensional
$\Theta(g(n))$	Orden de crecimiento asintótico ajustado superior e inferiormente	adimensional
α	Factor de carga n/m de una tabla hash	adimensional
$\phi(x_1, x_2)$	Función de selección en punto de convergencia SSA	adimensional
$n!$	Cardinalidad del grupo simétrico de permutaciones de n elementos	adimensional

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

Problema resuelto [PRB-0001]
Un vector dinámico comienza con capacidad inicial $\text{cap}_0 = 1$ y duplica su capacidad cada vez que se llena ($1 \to 2 \to 4 \to 8 \to 16 \to 32$). Si se insertan secuencialmente $n = 17$ elementos y cada inserción cuesta 1 escritura más el número de elementos copiados durante el redimensionado ($\$1 + 2 + 4 + 8 + 16\$$ copias acumuladas), calcule el coste total exacto de escrituras realizadas.
Respuesta

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[VOL02-C01-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 28
Ejercicio 2 [PRB-0004]
[VOL02-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 30
Ejercicio 3 [PRB-0005]
[VOL02-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 4 [PRB-0006]
[VOL02-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(14 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 5 [PRB-0007]
[VOL02-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 54
Ejercicio 6 [PRB-0008]
[VOL02-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 80
Ejercicio 7 [PRB-0009]

[VOL02-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 84
Ejercicio 8 [PRB-0010]
[VOL02-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 101
Ejercicio 9 [PRB-0011]
[VOL02-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 124
Ejercicio 10 [PRB-0012]
[VOL02-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 126
Ejercicio 11 [PRB-0013]
[VOL02-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 165
Ejercicio 12 [PRB-0014]
[VOL02-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 168

Cálculos reproducidos

$(17 + 31) = 48$

C.2 Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

Problema resuelto [PRB-0015]
Determine el número mínimo teórico de comparaciones binarias en el peor caso necesarias para ordenar $n = 5$ elementos distintos sabiendo que existen $5! = 120$ permutaciones posibles y que $2^6 = 64 < 120 \leq 2^7 = 128$, verificando además la diferencia $\$2^7 - 5!\$$.
Respuesta

8

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0017]
[VOL02-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 32

Ejercicio 2 [PRB-0018]

[VOL02-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 32

Ejercicio 3 [PRB-0019]

[VOL02-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 52

Ejercicio 4 [PRB-0020]

[VOL02-C02-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 48

Ejercicio 5 [PRB-0021]

[VOL02-C02-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 62

Ejercicio 6 [PRB-0022]

[VOL02-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 90

Ejercicio 7 [PRB-0023]

[VOL02-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 96

Ejercicio 8 [PRB-0024]

[VOL02-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 115

Ejercicio 9 [PRB-0025]

[VOL02-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 140

Ejercicio 10 [PRB-0026]

[VOL02-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 144

Ejercicio 11 [PRB-0027]

[VOL02-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 11, verifique las precon-

diciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 185
Ejercicio 12 [PRB-0028]
[VOL02-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 192

Cálculos reproducidos

$(128 - 120) = 8$

C.3 Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

Problema resuelto [PRB-0029]
En el algoritmo de Karatsuba para multiplicar dos enteros de n dígitos dividiendo cada operando en dos mitades, se sustituyen las 4 multiplicaciones clásicas de tamaño $n/2$ por solo 3 multiplicaciones recursivas. Compare el número de multiplicaciones base en una profundidad de $k = 4$ niveles ($n = 16$ bloques) entre el método cuadrático clásico ($\$4^4\$$) y Karatsuba ($\$3^4\$$), calculando el ahorro neto de productos elementales.
Respuesta

175

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0031]
[VOL02-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36
Ejercicio 2 [PRB-0032]
[VOL02-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 34
Ejercicio 3 [PRB-0033]
[VOL02-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 58
Ejercicio 4 [PRB-0034]
[VOL02-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 5 [PRB-0035]
[VOL02-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70

Ejercicio 6 [PRB-0036]
[VOL02-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 100
Ejercicio 7 [PRB-0037]
[VOL02-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 8 [PRB-0038]
[VOL02-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 129
Ejercicio 9 [PRB-0039]
[VOL02-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 156
Ejercicio 10 [PRB-0040]
[VOL02-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 162
Ejercicio 11 [PRB-0041]
[VOL02-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 205
Ejercicio 12 [PRB-0042]
[VOL02-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 216

Cálculos reproducidos

$(256 - 81) = 175$

C.4 Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

Problema resuelto [PRB-0043]
En una tabla hash con direccionamiento abierto y sondeo lineal que se mantiene con un factor de carga $\alpha = 0{,}75$, calcule el número esperado de sondeos en una búsqueda sin éxito (inserción de clave nueva) según la fórmula de Knuth $E_{\text{miss}} = \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$.

Respuesta

8,5

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0045]
[VOL02-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40

Ejercicio 2 [PRB-0046]
[VOL02-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36

Ejercicio 3 [PRB-0047]
[VOL02-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64

Ejercicio 4 [PRB-0048]
[VOL02-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56

Ejercicio 5 [PRB-0049]
[VOL02-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 78

Ejercicio 6 [PRB-0050]
[VOL02-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110

Ejercicio 7 [PRB-0051]
[VOL02-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120

Ejercicio 8 [PRB-0052]
[VOL02-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 143

Ejercicio 9 [PRB-0053]
[VOL02-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 172

Ejercicio 10 [PRB-0054]

[VOL02-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 180

Ejercicio 11 [PRB-0055]

[VOL02-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 225

Ejercicio 12 [PRB-0056]

[VOL02-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 240

Cálculos reproducidos

$(0{,}5 \cdot (1 + 16)) = 8{,}5$

C.5 Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

Problema resuelto [PRB-0057]

Un algoritmo paralelo de reducción y filtrado tiene un trabajo total $T_1 = 9600$ ms y un camino crítico (span) $T_\infty = 40$ ms. Calcule la cota superior garantizada de tiempo de ejecución $T_P = T_1/P + T_\infty$ en $P = 16$ procesadores bajo un planificador voraz.

Respuesta

640

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0059]

[VOL02-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 44

Ejercicio 2 [PRB-0060]

[VOL02-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 38

Ejercicio 3 [PRB-0061]

[VOL02-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 70

Ejercicio 4 [PRB-0062]

[VOL02-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 5 [PRB-0063]
[VOL02-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 86
Ejercicio 6 [PRB-0064]
[VOL02-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 7 [PRB-0065]
[VOL02-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 8 [PRB-0066]
[VOL02-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 157
Ejercicio 9 [PRB-0067]
[VOL02-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 188
Ejercicio 10 [PRB-0068]
[VOL02-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 198
Ejercicio 11 [PRB-0069]
[VOL02-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 245
Ejercicio 12 [PRB-0070]
[VOL02-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 264

Cálculos reproducidos

$(600 + 40) = 640$

C.6 Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

Problema resuelto [PRB-0071]
Calcule el ahorro total en kilobytes (1 KiB = 1024 bytes) de un vector en memoria de $N = 65536$ registros cuando se reordena en C una estructura mal alineada de 24 bytes (`uint8_t`, 7 bytes de padding, `uint64_t`, `uint8_t`, 7 bytes de padding) a su disposición óptima de 16 bytes (`uint64_t`, `uint8_t`, `uint8_t`, 6 bytes de padding final).
Respuesta

512

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0073]
[VOL02-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 48
Ejercicio 2 [PRB-0074]
[VOL02-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 3 [PRB-0075]
[VOL02-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 76
Ejercicio 4 [PRB-0076]
[VOL02-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(24 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64
Ejercicio 5 [PRB-0077]
[VOL02-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 94
Ejercicio 6 [PRB-0078]
[VOL02-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 130
Ejercicio 7 [PRB-0079]
[VOL02-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 144
Ejercicio 8 [PRB-0080]

[VOL02-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 171
Ejercicio 9 [PRB-0081]
[VOL02-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 204
Ejercicio 10 [PRB-0082]
[VOL02-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 216
Ejercicio 11 [PRB-0083]
[VOL02-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 265
Ejercicio 12 [PRB-0084]
[VOL02-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 288

Cálculos reproducidos

$\frac{(65536 \cdot 8)}{1024} = 512$

C.7 Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

Problema resuelto [PRB-0085]
En un bloque básico antes de la optimización se evalúa en un bucle de $N = 1000$ iteraciones una expresión con una subexpresión invariante de bucle de coste $c_{\text{inv}} = 6$ ciclos y una parte dependiente del índice de coste $c_{\text{var}} = 3$ ciclos por iteración. Calcule el ahorro total de ciclos al aplicar el pase de Movimiento de Código Invariante de Bucle (LICM), que evalúa la subexpresión invariante una única vez fuera del bucle.
Respuesta

5994

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0087]
[VOL02-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 2 [PRB-0088]
[VOL02-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 2, verifique las

precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 42
Ejercicio 3 [PRB-0089]
[VOL02-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 82
Ejercicio 4 [PRB-0090]
[VOL02-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(26 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 68
Ejercicio 5 [PRB-0091]
[VOL02-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 102
Ejercicio 6 [PRB-0092]
[VOL02-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 140
Ejercicio 7 [PRB-0093]
[VOL02-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 156
Ejercicio 8 [PRB-0094]
[VOL02-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 185
Ejercicio 9 [PRB-0095]
[VOL02-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 220
Ejercicio 10 [PRB-0096]
[VOL02-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 234
Ejercicio 11 [PRB-0097]
[VOL02-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 285
Ejercicio 12 [PRB-0098]

[VOL02-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 312

Cálculos reproducidos

$(9000 - 3006) = 5994$

C.8 Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

Problema resuelto [PRB-0099]
En una arquitectura con páginas de 4 KiB = 4096 bytes (2^{12} bytes), dada la dirección virtual decimal $V = 20500$, determine el número de página virtual VPN = $\lfloor V/4096 \rfloor$, el desplazamiento dentro de la página offset = $V - \text{VPN} \cdot 4096$, y calcule la dirección física resultante si la tabla de páginas asigna dicho VPN al marco físico PFN = 12.
Respuesta

49172

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0101]
[VOL02-C08-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 2 [PRB-0102]
[VOL02-C08-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 3 [PRB-0103]
[VOL02-C08-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 94
Ejercicio 4 [PRB-0104]
[VOL02-C08-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(30 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 76
Ejercicio 5 [PRB-0105]
[VOL02-C08-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 118
Ejercicio 6 [PRB-0106]

[VOL02-C08-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 160
Ejercicio 7 [PRB-0107]
[VOL02-C08-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180
Ejercicio 8 [PRB-0108]
[VOL02-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 213
Ejercicio 9 [PRB-0109]
[VOL02-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 252
Ejercicio 10 [PRB-0110]
[VOL02-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 270
Ejercicio 11 [PRB-0111]
[VOL02-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 325
Ejercicio 12 [PRB-0112]
[VOL02-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 360

Cálculos reproducidos

$((12 \cdot 4096) + 20) = 49172$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 25 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios. Ecuaciones: 8 respaldada por una fuente, 11 valor o condición.
¶3: dos testimonios. Ecuaciones: 3 verificada simbólicamente, 1 reproducida numéricamente, 10 respaldada por una fuente, 2 valor o condición.
¶4: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶5: dos testimonios.
¶6: dos testimonios. Ecuaciones: 1 valor o condición.
¶7: dos testimonios.
¶8: dos testimonios. Ecuaciones: 4 valor o condición.
¶9: dos testimonios. Ecuaciones: 2 respaldada por una fuente, 2 valor o condición.
¶10: dos testimonios. Ecuaciones: 2 respaldada por una fuente, 1 valor o condición.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios. Ecuaciones: 1 aproximación.
¶15: dos testimonios.
¶16: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶22: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶27: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 aproximación, 4 valor o condición.
¶28: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

MIT OpenCourseWare 6.006 — Introduction to Algorithms (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
The Rust Programming Language (Klabnik & Nichols — Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
E. F. Codd — A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
C Programming Language Reference (ISO/IEC 9899 C17/C23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
C++ Reference (ISO/IEC 14882 C++20/C++23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Rust Reference — Memory Model, Type System, Lifetimes, and Unsafe Rust (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-complejidad-asintotica-recurrencias-amortizado scientific
REV-0002: cap02-busqueda-ordenacion-cotas-inferiores scientific
REV-0003: cap03-divide-venceras-voraces-programacion-dinamica scientific
REV-0004: cap04-grafos-arboles-equilibrados-hashing scientific
REV-0005: cap05-estructuras-persistentes-probabilisticas-paralelos scientific
REV-0006: cap06-jerarquia-memoria-caches-abi-enlazado scientific
REV-0007: cap07-compiladores-lexing-parsing-ssa-codegen scientific
REV-0008: cap08-sistemas-operativos-procesos-memoria-virtual-io scientific

Apéndice E

Plan de estudio

PARTE III. Algoritmos, Estructuras de Datos y Computación

✓Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

- Dominar Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

2 conceptos planificados.

✓Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

- Dominar Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

2 conceptos planificados.

✓Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

- Dominar Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

2 conceptos planificados.

✓Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

- Dominar Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

2 conceptos planificados.

✓Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

- Dominar Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

2 conceptos planificados.

PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos

✓Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

- Dominar Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

2 conceptos planificados.

✓Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

- Dominar Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

2 conceptos planificados.

✓Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

- Dominar Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering